

RSE - Robustness Laboratory

Elevator

Introduction

The aim of this laboratory is to optimize an elevator control system so that it meets current safety requirements. A first version of the elevator control system is available. This allows the elevator to move from one floor to another (the elevator only has two floors). What is missing, however, are control mechanisms to ensure the safety of passengers in the event of sensor or mechanical failure.

The goal is to extend this software to such an extent that the elevator stops as soon as the elevator malfunctions.

Prerequisites

Hardware

You will be provided with a miniature elevator:

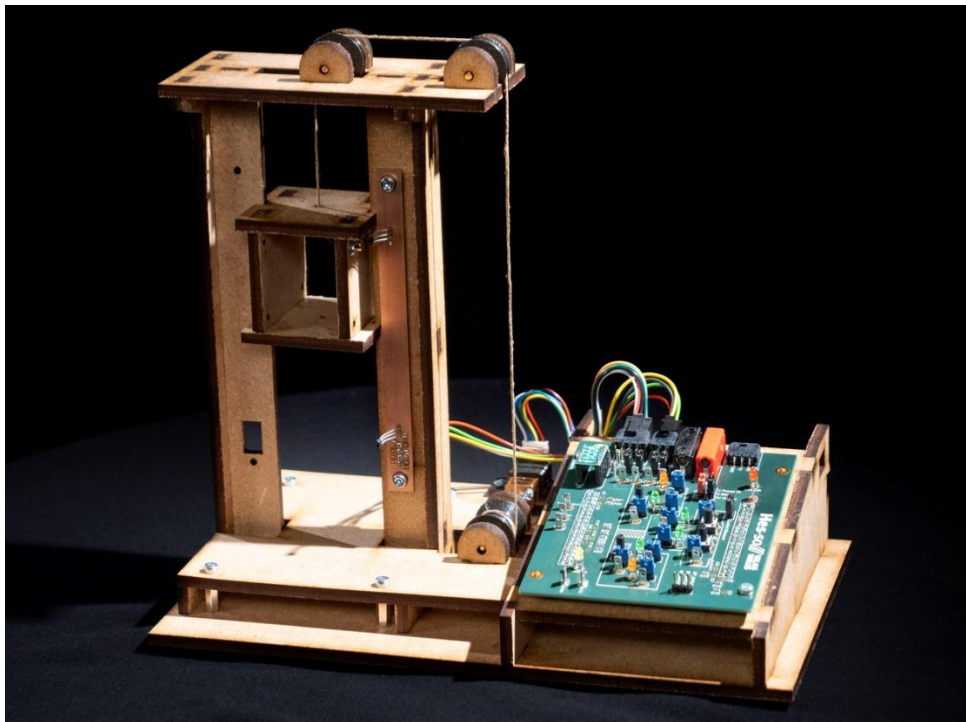


Figure 1: Miniature Elevator

The embedded system software runs on a Nucleo board based on an STM32L476RG which is connected underneath the control panel (green PCB).

To download, debug and trace the software on the Nucleo board, you need to connect it to your development PC using a USB cable.

Development Tools

VSCode is used to develop, compile, download and debug the code.

To receive and show the messages send over the *virtual com port* (VCP) (USB cable), you can use CoolTerm or any other terminal software able to communicate over a serial port.

Tool	Comment
 VSCode	To develop the <i>ButtonManager</i> Application for the Embedded System and to configure and generate the code for the MCU peripherals.
 CoolTerm	To log text output of the Embedded System on the development PC.

Software

On the moodle server in the corresponding laboratory section, you will find in the folder *Files/work* a file named **work.zip**. It provides the following files and additional information:

Folder	Comment
docs	Folder with documentation
ide-vscode	VSCode based <i>Elevator</i> project
src/app	<i>Elevators</i> application code
src/config	Header files to configure packages
src/event	Custom XF events for this project
src/mdw	Middleware package (with <i>trace</i> package)
src/platform	Platform and board specific code
src/security	Security package to monitor the elevator controller
src/xf	Execution Framework used in this project (RSE XF)

Block Diagram

Below you can find a block diagram of the elevator:

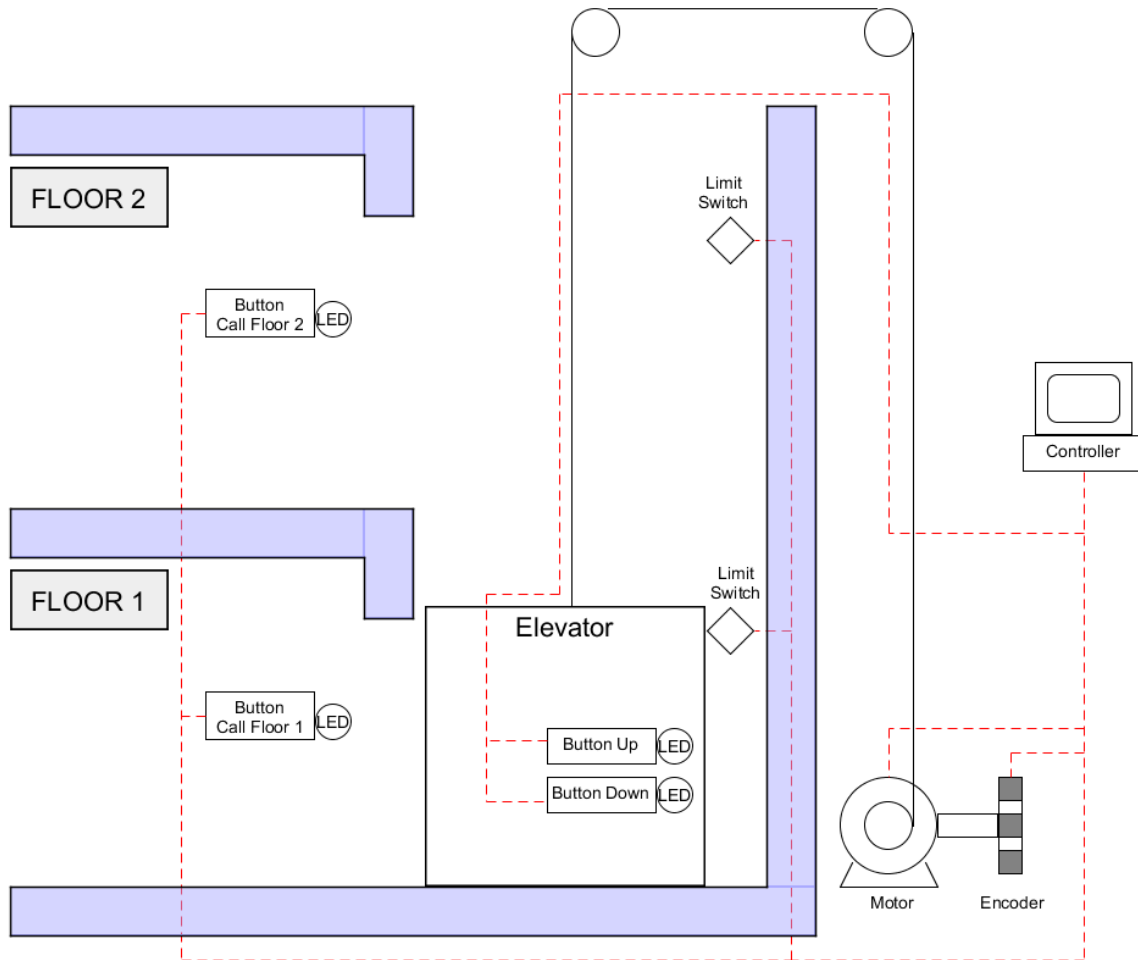


Figure 2: Elevator Block Diagram

On each floor the elevator can be called using a button. Inside the elevator are two buttons to move the elevator either up or down. Each button has its own indicator LED.

The elevator is moved using a motor. The position of the elevator is monitored with an encoder mounted on the motors shaft.

The presents of the elevator on a floor gets detected using a limit switch.

Elevator Control Application

Elevator Control Class Overview

Elevator Controller Class

The `elevator::Controller` class is responsible to move the elevator from one floor to another.

The elevators motor is driven using the `motor::Driver` class. The `elevator::Controller` is the only class allowed to use the `motor::Driver`:

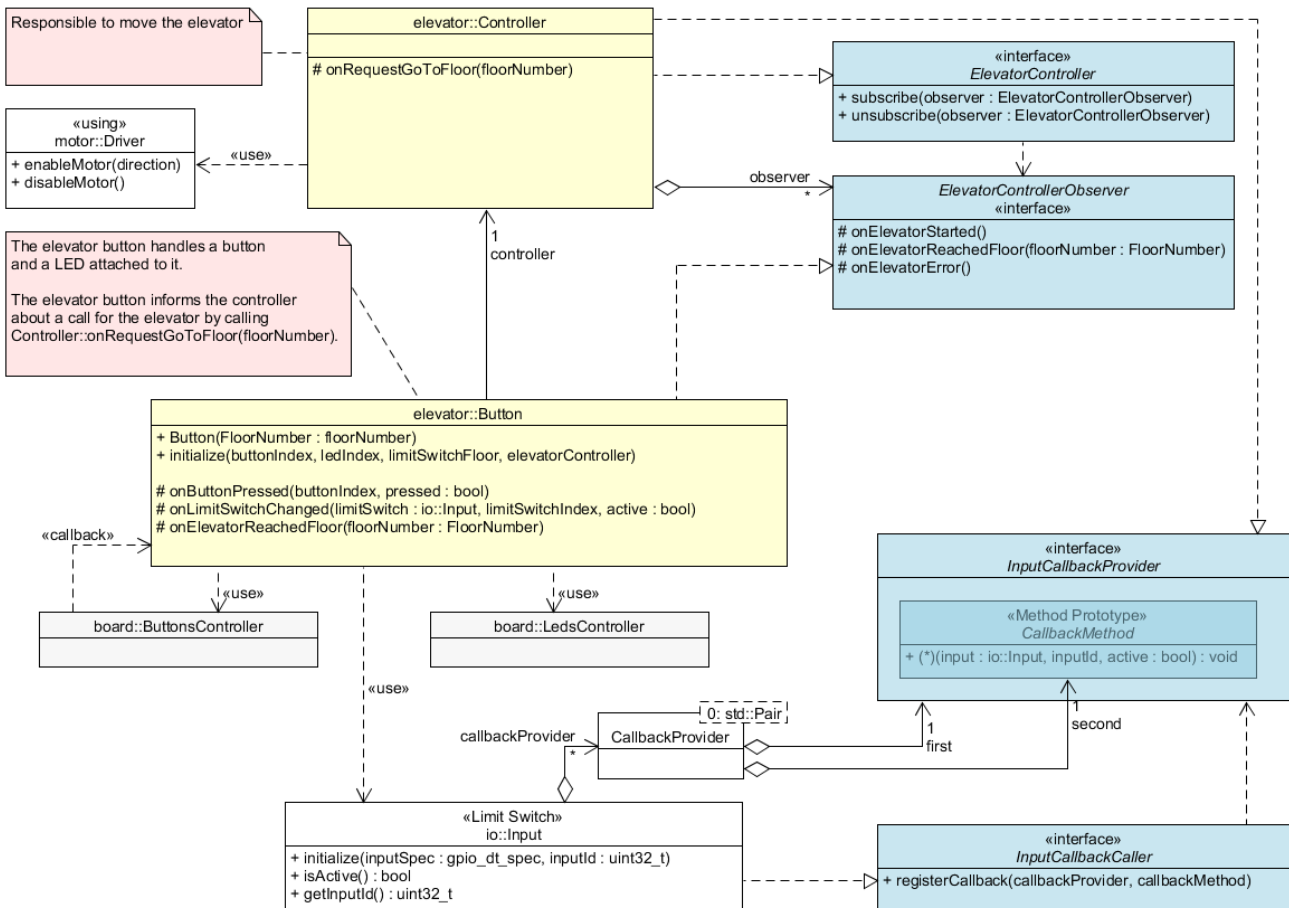


Figure 3: Overview: Elevator Controller Class Diagram

The `elevator::Controller` can be observed using the `ElevatorControllerObserver` interface.

Elevator Button Class

The `elevator::Button` class handles a physical button together with its corresponding LED. It may represent a button on the floor or a button inside the elevator. The `elevator::Button` memorises the event when a user presses the button (LED is light up) and notifies the `elevator::Controller` about the event. It observes its corresponding limit switch (`io::Input`) to detect the arrival of the elevator and to clear the elevator call (and the LED).

Limit Switch – io::Input Class

The presents of the elevator on a floor gets detected using a hall sensor. The hall sensor is connected via an input pin to the Microcontroller. The `io::Input` class detects level changes on the input pin using interrupts, debounces the signal and communicates level changes using a method callback pattern.

Elevator Controller State Machine Diagram

Following you will find a state machine diagram representing roughly the behaviour of the `elevator::Controller` class:

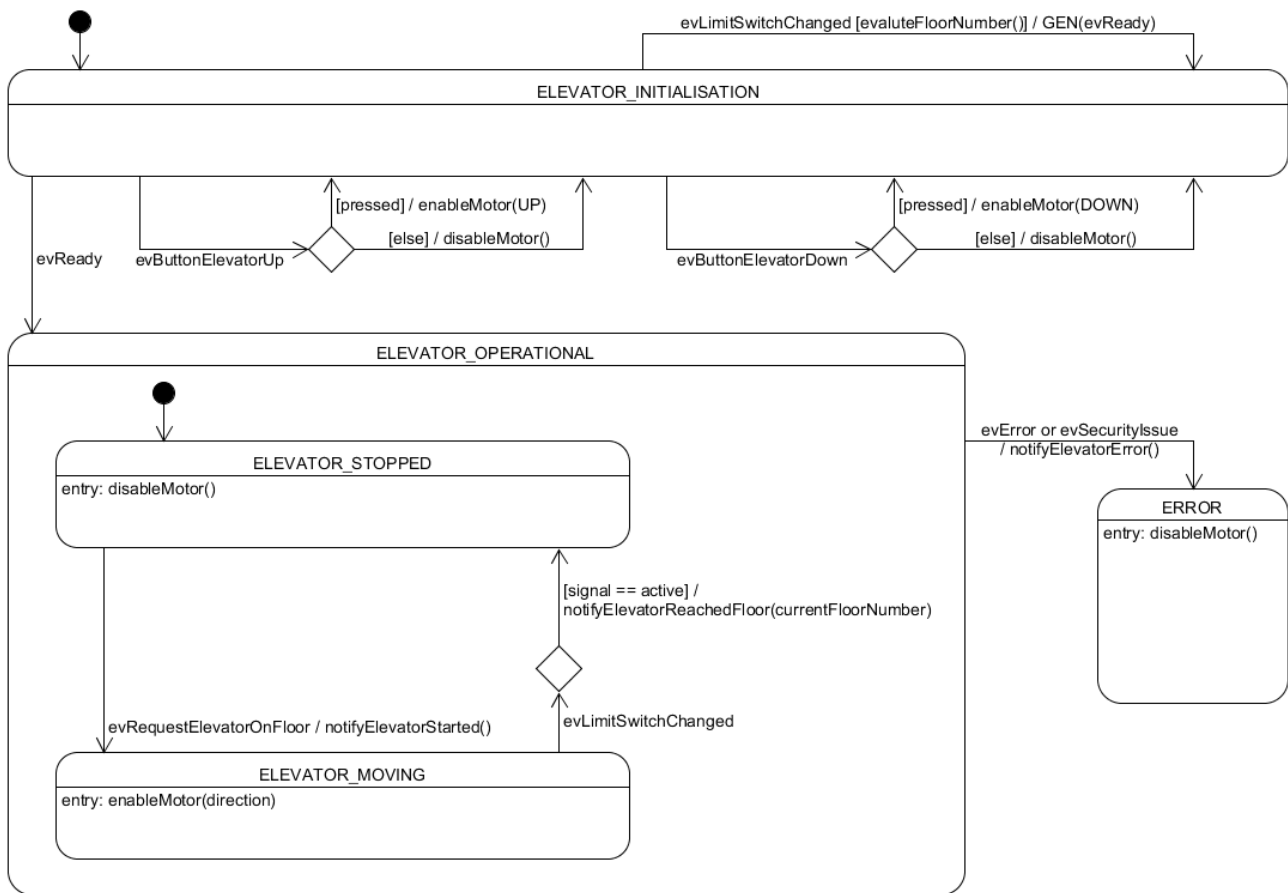


Figure 4: Elevator Controller State Machine Diagram

The elevator needs first to be initialised to a specific floor. During this time the elevator can be used using the buttons. This is done using the *INITIALISATION* state.

Once a floor is reached the elevator goes into *OPERATIONAL* state. From there the elevator can be called to a floor by pressing the corresponding buttons.

In case an error is detected, the elevator state machine goes into *ERROR* state.

Elevator Factories Overview

The elevator application uses the `app::Factory` to construct and initialize the objects for the elevator control part:

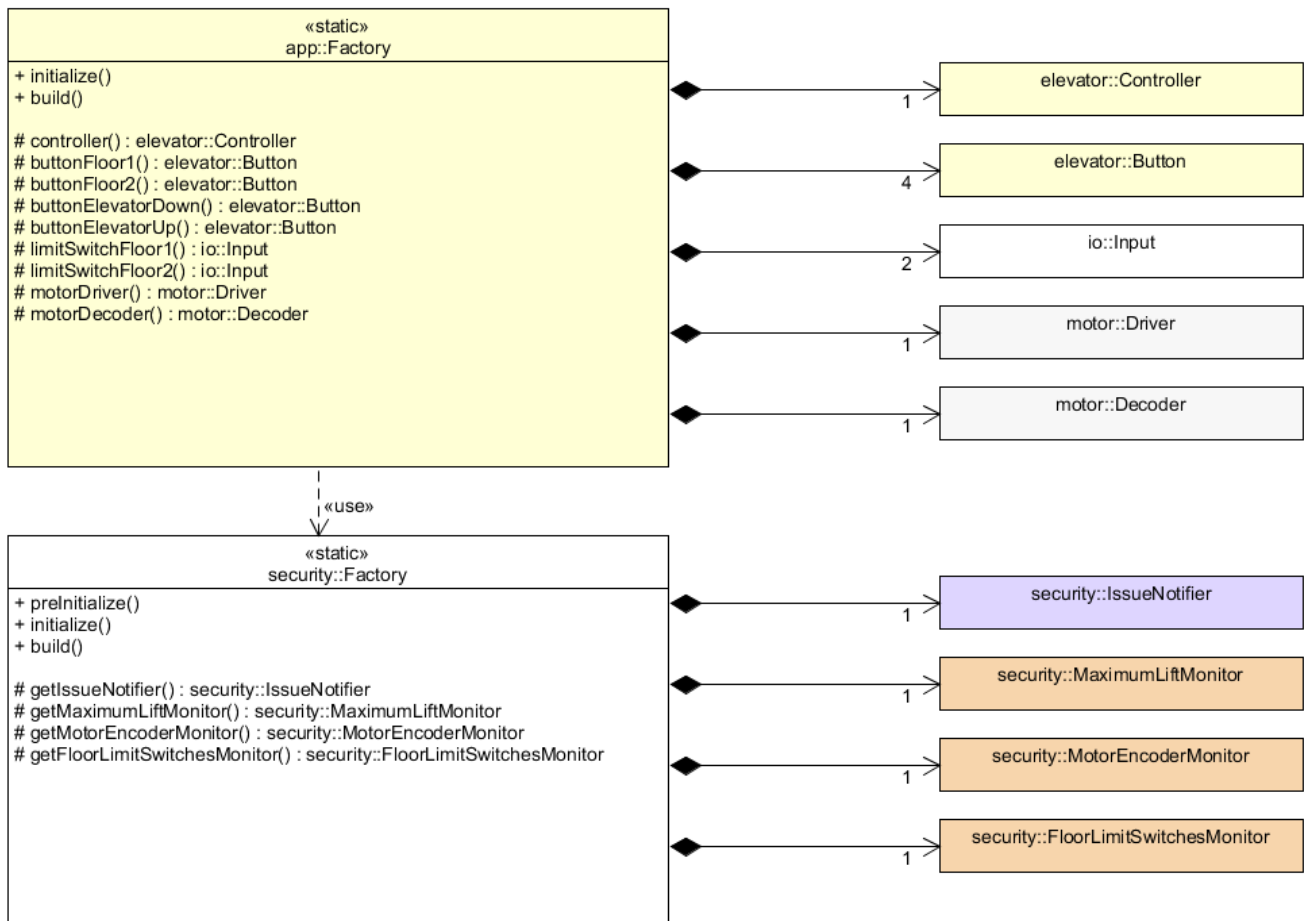


Figure 5: Overview: Elevator Factories Class Diagram

The `security::Factory` class is in charge to construct and initialize the security components used to monitor the elevator.

Software Robustness

Security Software Package

Using the security package, you can monitor parts of the software. In case a malfunction is detected, an issue is reported to the subscribed observers.

Each functionality to be monitored should be put into different classes derived from the `security::Monitor` class.

The `security::Monitor` serves as common interface to communicate a security issue using the `security::IssueNotifier`.

Every class interested about security issues can subscribe to the `security::IssueNotifier` using the `SecurityIssueObserver` interface.

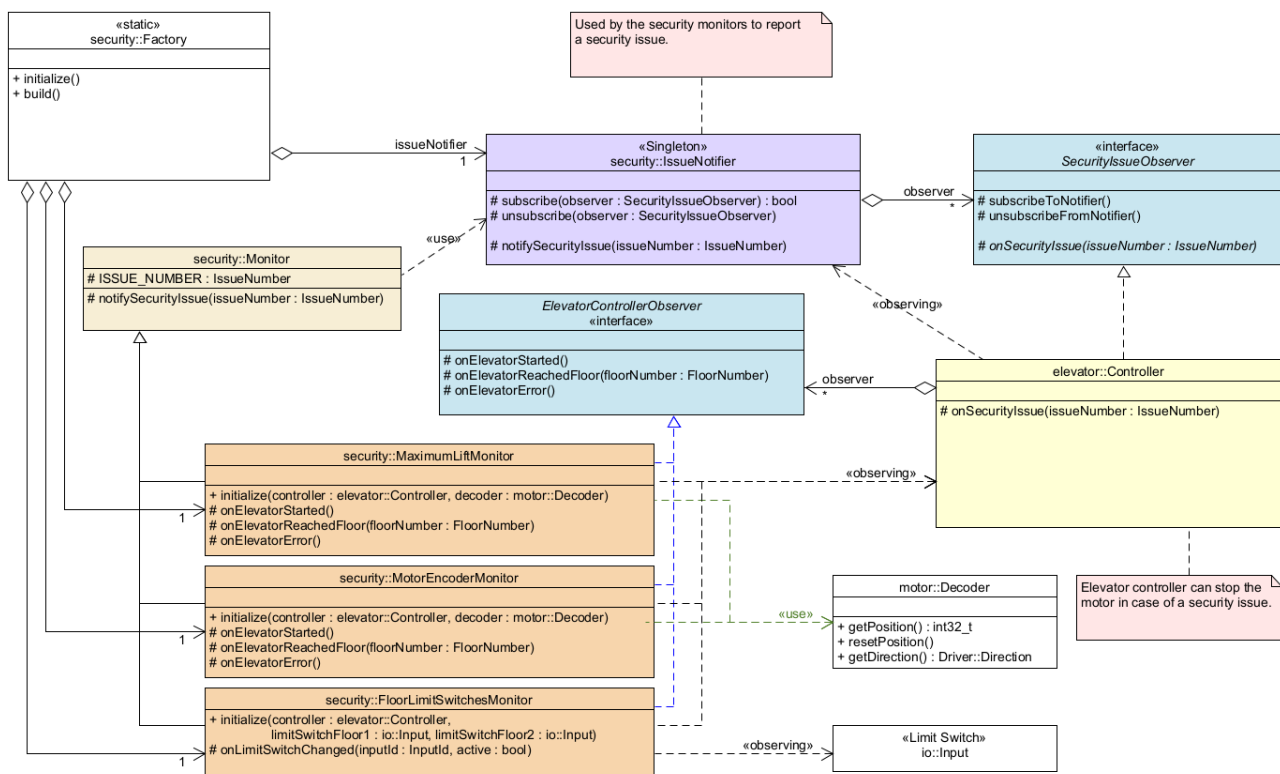


Figure 6: Overview: Security Package

The class diagram above shows three monitor classes (`security::MaximumLiftMonitor`, `security::MotorEncoderMonitor` and `security::FloorLimitSwitchesMonitor`). Each of these classes observe a different behaviour of the software.

The `elevator::Controller` subscribes to the `security::IssueNotifier` to stop the elevator in case of an issue.

Issue Notification Procedure

Following sequence diagram shows how a *security monitor* is notifying a security issue to the subscribed *security issue observers*. In this example the `security::MaximumLiftMonitor` is notifying an issue to the `elevator::Controller`:

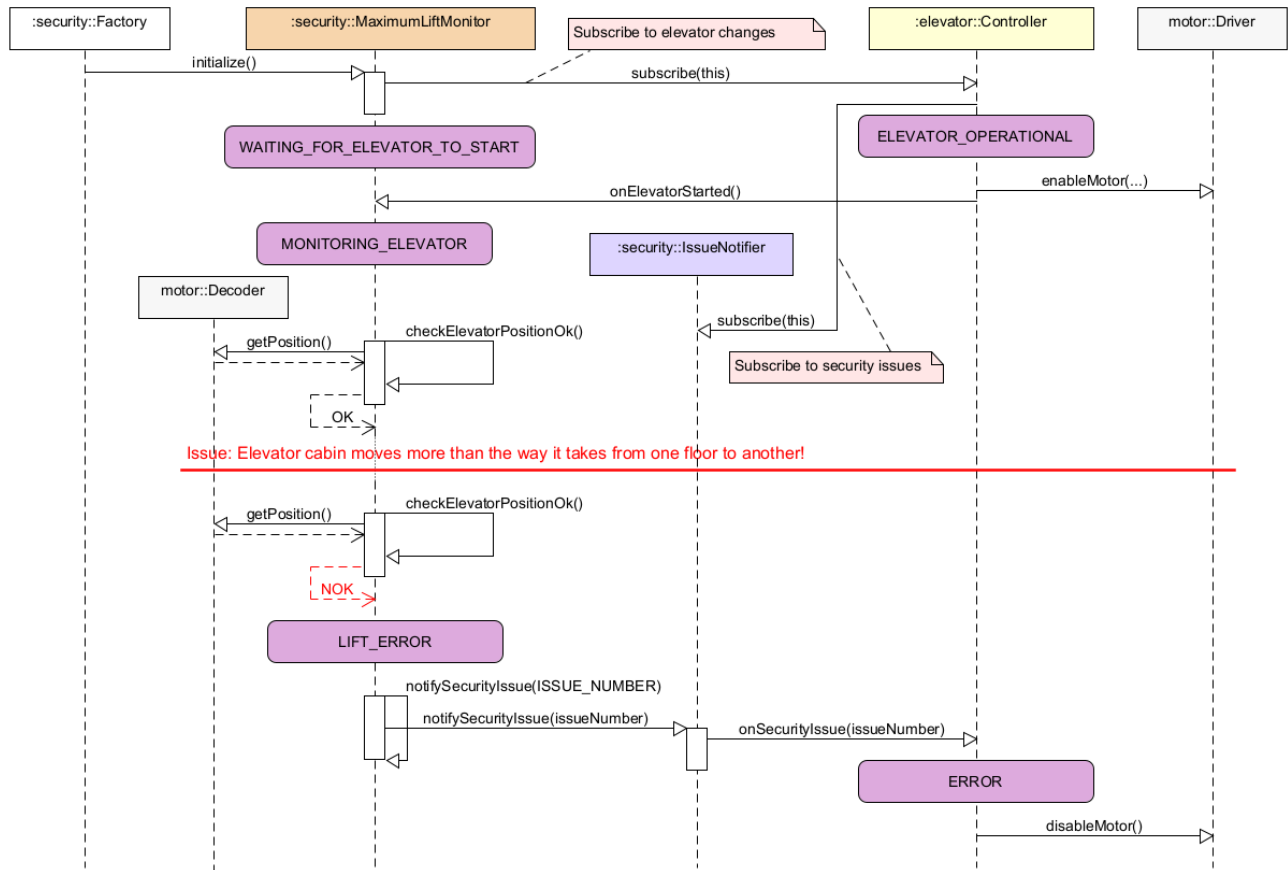


Figure 7: Sequence Diagram: Security Issue Notification

Task 1 – MaximumLiftMonitor Implementation

Add a security monitor supervising the motors movement. In case a limit-switch on a floor is defect, the monitor needs to inform the controller to stop the elevator.

1. Develop scenarios for this case and write them down using UML sequence diagrams.
2. Implement the `security::MaximumLiftMonitor` based on the developed scenarios.
3. Blackbox test the `security::MaximumLiftMonitor` class and write down the test results in the report.

The following diagram shall give an idea on how to implement the state machine for the monitor class:

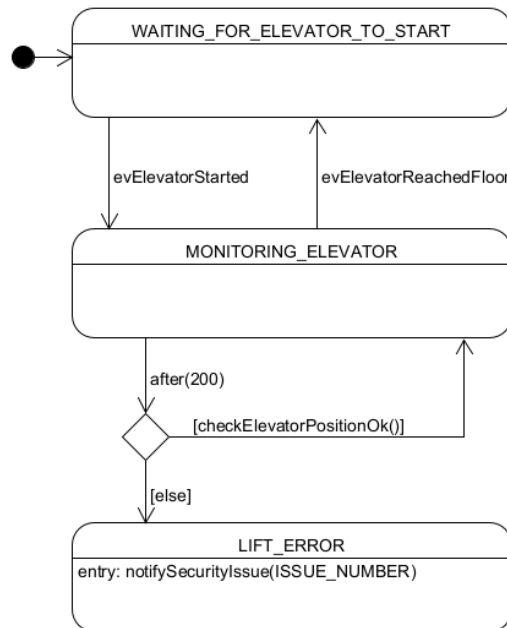


Figure 8: Example State Machine Diagram: MaximumLiftMonitor

Task 2 – MotorEncoderMonitor Implementation

1. Add a security monitor ensuring the proper functioning of the motor. In case the motor gets stuck or in general stops turning during the time it passes from one floor to another, the monitor needs to inform the controller to stop the elevator.
2. Develop scenarios for this case and write them down using UML sequence diagrams.
3. Implement the `security::MotorEncoderMonitor` based on the developed scenarios.
4. Blackbox test the `security::MotorEncoderMonitor` class and write down the test results in the report.

Also here an example diagram how you might implement the state machine:

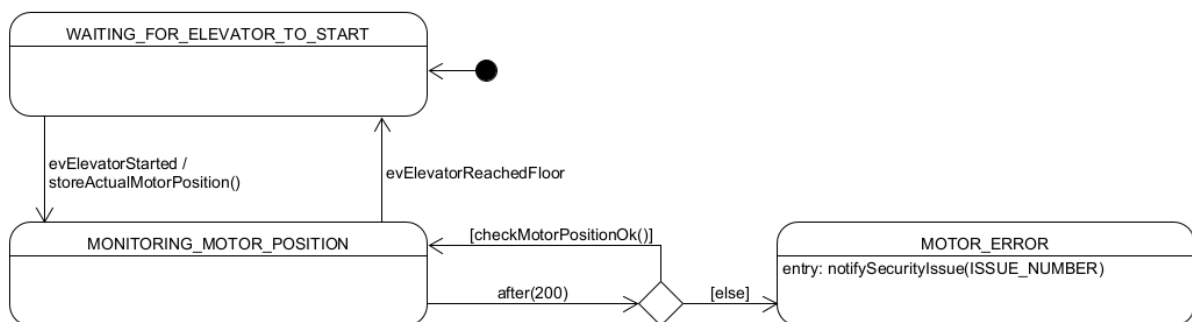


Figure 9: Example State Machine Diagram: MotorEncoderMonitor

Task 3 – FloorLimitSwitchesMonitor Implementation

1. Add a security monitor that keeps track of the limit switches on all floors.
2. Develop scenarios for this case and write them down using UML sequence diagrams.
3. Provide a state machine diagram to implement the designed behavior
4. Implement the `security::FloorLimitSwitchesMonitor` based on the developed scenarios.
5. Blackbox test the `security::FloorLimitSwitchesMonitor` class and write down the test results in the report.

Task 4 – To-do's and Questions

Following you will find some tasks and questions to deepen your understanding of the actual elevator system:

1. Draw a system diagram (block diagram, processes are clouds) of the different processes of the elevator system. Document which messages are exchanged between these processes and which importance these messages have for the robustness of the Elevator system
2. How would you solve the following problem: A non-important message arrives just in front of a very important message and delays this latter's immediate treatment. The two messages are sent by different other processes
3. Propose at least two more processes that potentially could still improve the robustness of the elevator system
4. By which means are the processes of the Elevator system realised?
5. By which means does the XF (used to implement the state machines in the elevator firmware) contribute to the robustness of the entire system?
6. Which software patterns are used to develop the Elevator system and how do these patterns contribute to the robustness of the system?
7. Which of the software parts developed by yourself did you white box test and how?
8. Show at least two **C++ do's** that you applied in your code
9. Show at least two **C++ dont's** you omitted in your code and how you work around them

Delivery

Deposit on the Moodle server a zip file containing following information:

- Your *Elevator* project
- A report (PDF file) showing your realised work:
 - o The sequence diagrams for the tasks 1 to 3
 - o Test results for the tasks 1 to 3
 - o Your results and answers for task 4